

```
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define MAXPAROLA 30
#define MAXRIGA 80
```

```
int main(int argc, char *argv[])
{
    int freq[MAXPAROLA]; /* vettore di contatori
delle frequenze delle lunghezze delle parole */
    char riga[MAXRIGA];
    int i, inizio, lunghezza;
    FILE *f;
```

```
for(i=0; i<MAXPAROLA; i++)
    freq[i]=0;
```

```
if(argc != 2)
```

```
{
    fprintf(stderr, "ERRORE, serve un parametro con il nome del file\n");
    exit(1);
}
```

```
f = fopen(argv[1], "r");
if(f==NULL)
```

```
{
    fprintf(stderr, "ERRORE, impossibile aprire il file %s\n", argv[1]);
    exit(1);
}
```

```
while( fgets( riga, MAXRIGA, f ) != NULL )
```



Advanced IO

File Locking

Stefano Quer

Dipartimento di Automatica e Informatica

Politecnico di Torino

License Information

This work is licensed under the license



Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

① **BY:** Credit must be given to you, the creator.

② **NC:** Only noncommercial use of your work is permitted.

Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

③ **ND:** No derivatives or adaptations of your work are permitted.

To view a copy of the license, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/?ref=chooser-v1>

File locking

- ❖ When two tasks edit the same file at the same time the final state of the file corresponds to the last task that wrote the file
 - Several applications need to be certain that are the only one to write to a file
- ❖ It is always possible to consider a file as a critical section and to protect its access with a lock
 - However, a file can be easily protected **entirely** or **not at all**
 - This approach limits concurrency
 - Partial protection is difficult to grant with standard techniques

Pay attention on whether accessing the file with a single or multiple file descriptor

File locking

- ❖ There are applications that would be more efficient protecting only **sections** of a file
 - Protecting file sections would indeed allow different tasks to work in parallel on the file
- ❖ To grant partial file protection, several OSs provide **file locking**
 - It is the term normally used to describe the ability of a process to prevent other processes from modifying a **region** of a file
 - File locking is a limited form of **synchronization**

The commands

```
int fcntl (int fd, int cmd, struct flock *flockptr);
```

- ❖ In UNIX-POSIX, file locking is performed as a byte-range locking
 - A range of a file (possibly, the entire file) is locked
- ❖ For locking we use function **fcntl**
 - The parameter **cmd** must be set to
 - F_GETLK, F_SETLK, or F_SETLKW
 - The third argument must point to a **flock structure**

The commands

```
int fcntl (int fd, int cmd, struct flock *flockptr);
```



cmd	Purpose
F_GETFL or F_SETFL	Get/set file status flags
F_DUPFD or F_DUPFD_CLOEXEC	Duplicate an existing descriptor
F_GETFD or F_SETFD	Get/set file descriptor flags
F_GETOWN or F_SETOWN	Get/set asynchronous I/O ownership
F_GETLK, F_SETLK, or F_SETLKW	Get/set record locks

Three commands can be issued

The commands

```
int fcntl (int fd, int cmd, struct flock *flockptr);
```



cmd	Purpose
F_GETLK , F_SETLK, F_SETLKW	Get/set record locks

➤ F_GETLK

- **Check** whether the lock described by **flockptr** is blocked by some other lock
 - If a lock exists that would prevent ours from being created, the information on that existing lock overwrites the information pointed to by flockptr

The commands

```
int fcntl (int fd, int cmd, struct flock *flockptr);
```



cmd	Purpose
F_GETLK, F_SETLK , F_SETLKW	Get/set record locks

➤ F_SETLK

- Set the lock described by flockptr
 - If the compatibility rule prevents the system from giving us the lock fcntl returns immediately with errno set to either EACCES or EAGAIN
 - This command is also used to clear the lock described by flockptr (l_type of F_UNLCK)

The commands

```
int fcntl (int fd, int cmd, struct flock *flockptr);
```



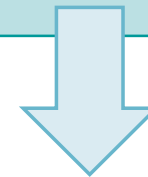
cmd	Purpose
F_GETLK, F_SETLK, F_SETLKW	Get/set record locks

➤ F_SETLKW

- The blocking version of F_SETLK
 - The W in the command name means wait
 - Set the lock described by flockptr with a blocking operation
- If the lock cannot be granted, the calling process is put to sleep and the process wakes up when the lock becomes available

The flock structure

```
int fcntl (int fd, int cmd, struct flock *flockptr);
```



A shared read lock F_RDLCK
An exclusive write lock F_WRLCK
Unlocking a region F_UNLCK

Offset in bytes
Relative to l_whence

Length, in bytes
0 means lock to EOF

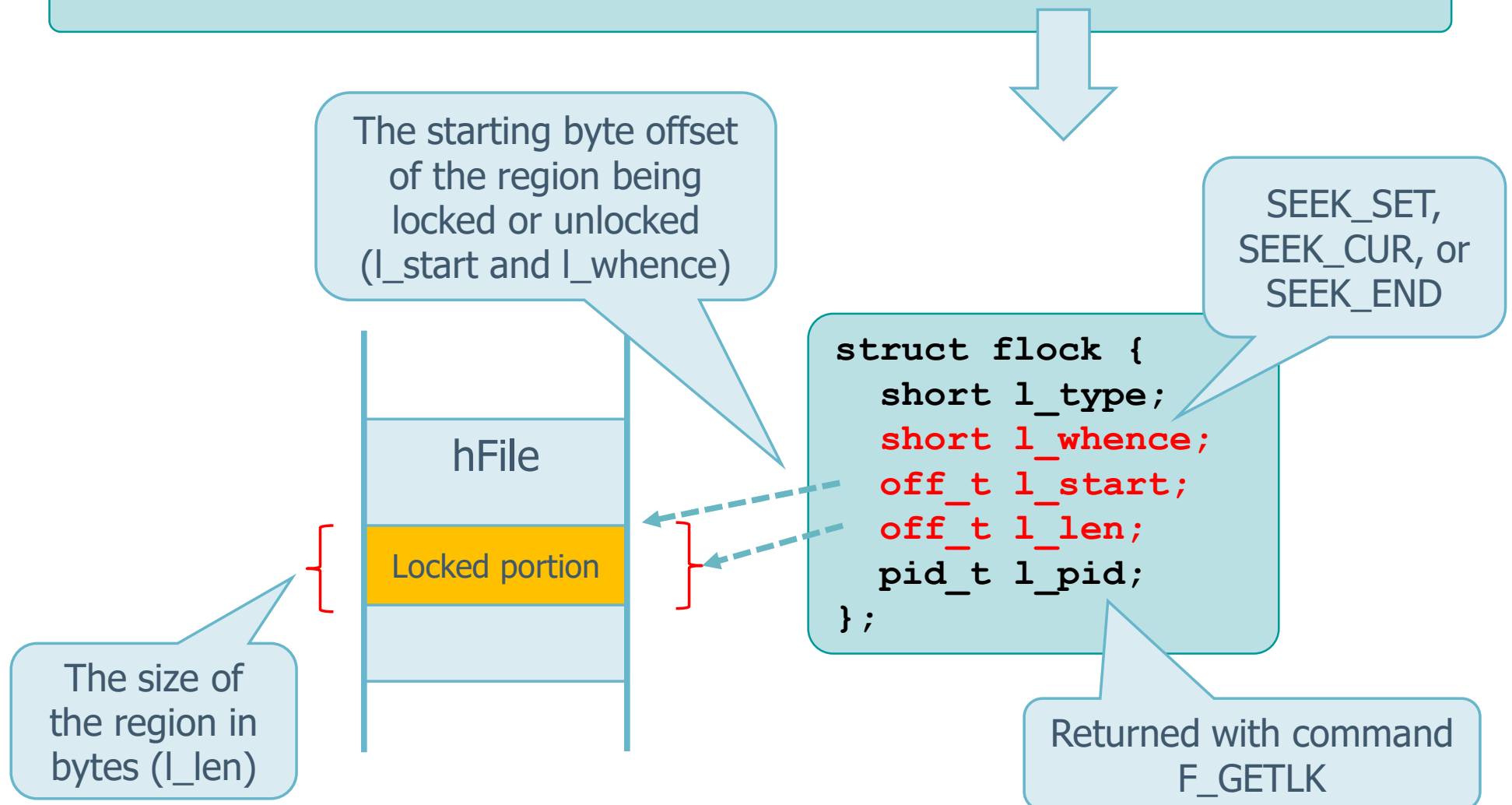
```
struct flock {  
    short l_type;  
    short l_whence;  
    off_t l_start;  
    off_t l_len;  
    pid_t l_pid;  
};
```

SEEK_SET,
SEEK_CUR, or
SEEK_END

Returned with F_GETLK

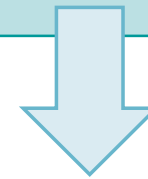
The flock structure

```
int fcntl (int fd, int cmd, struct flock *flockptr);
```



The flock structure

```
int fcntl(int fd, int cmd, struct flock *flockptr);
```



The ID (l_pid) of the process holding the lock that can block the current process (returned by F_GETLK only)

```
struct flock {  
    short l_type;  
    short l_whence;  
    off_t l_start;  
    off_t l_len;  
    pid_t l_pid;  
};
```

SEEK_SET,
SEEK_CUR, or
SEEK_END

Returned with command
F_GETLK

Guidelines

❖ Several rules apply

- The two parameters specifying the starting offset of the region are similar to the last two arguments of the **lseek** function

```
off_t lseek (int fd, off_t offset, int whence);
```

- Locks can start and extend beyond the current end of file, but cannot start or extend before the beginning of the file

```
struct flock {  
    short l_type;  
    short l_whence;  
    off_t l_start;  
    off_t l_len;  
    pid_t l_pid;  
};
```

Guidelines

- If **l_len** is 0, the lock extends to the largest possible offset of the file
 - This allows us to lock a region starting anywhere in the file, up through and including any data that is appended to the file
- To lock the entire file, we set **l_start** and **l_whence** to point to the beginning of the file and specify a length (**l_len**) of 0

```
struct flock {  
    short l_type;  
    short l_whence;  
    off_t l_start;  
    off_t l_len;  
    pid_t l_pid;  
};
```

Guidelines

➤ Repeated Lock Request

- If a lock is present
- A new lock request is granted or refused ?

Exiting Lock	Requested Lock Type	
	Read Lock	Write Lock
None	Granted	Granted
Read lock	Granted	Refused
Write lock	Refused	Refused

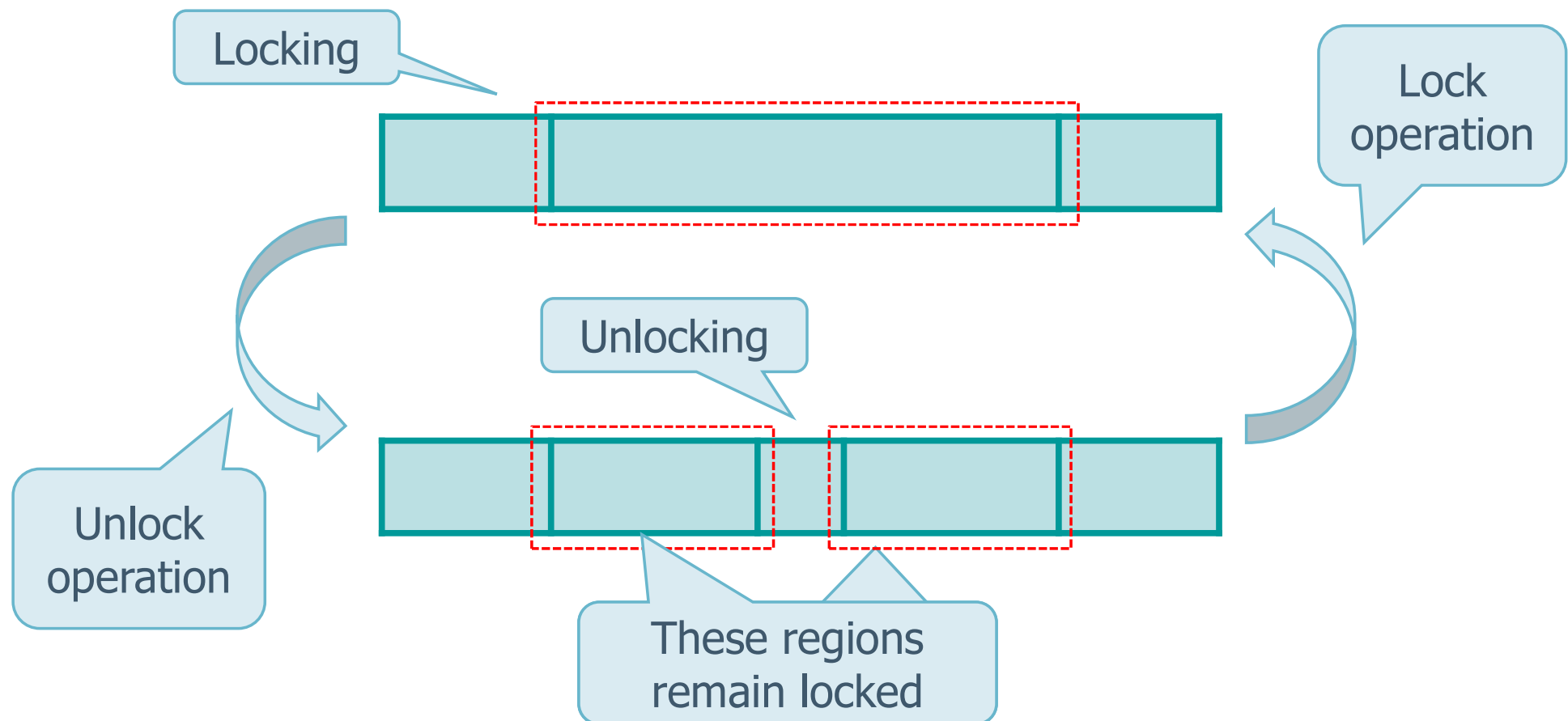
Guidelines

➤ Locks **belong** to a **processes**

- The compatibility rule applies to lock requests made from different processes, not to multiple lock requests made by a single process
- If a process has an existing lock on a range of a file, a subsequent attempt to place a lock on the same range by the same process will replace the existing lock with the new one
 - If a process has a write lock on bytes 16–32 of a file and then tries to place a read lock on bytes 16–32, the request will succeed, and the write lock will be replaced by a read lock

Guidelines

- When setting or releasing a lock on a file, the system combines or splits adjacent areas as required



Guidelines

➤ File locking can produce

- Starvation
 - Processes A and B periodically obtain a shared lock whereas C is waiting forever for an exclusive lock
- Deadlock
 - Process A is waiting for B to unlock and vice-versa (even on slightly different file regions)

Example

Lock or unlock a file region

```
#include <fcntl.h>
```

```
...
```

```
int lock_region (int fd, int cmd, int type, off_t offset,  
                int whence, off_t len) {
```

```
    struct flock lock;
```

```
    lock.l_type = type;  
    lock.l_start = offset;  
    lock.l_whence = whence;  
    lock.l_len = len;
```

```
    return (fcntl(fd, cmd, &lock));
```

```
}
```

Offset in bytes, relative to l_whence

A shared read lock F_RDLCK
An exclusive write lock F_WRLCK
Unlocking a region F_UNLCK

SEEK_SET, SEEK_CUR, or SEEK_END

Length, in bytes
0 means lock to EOF

-1 on error